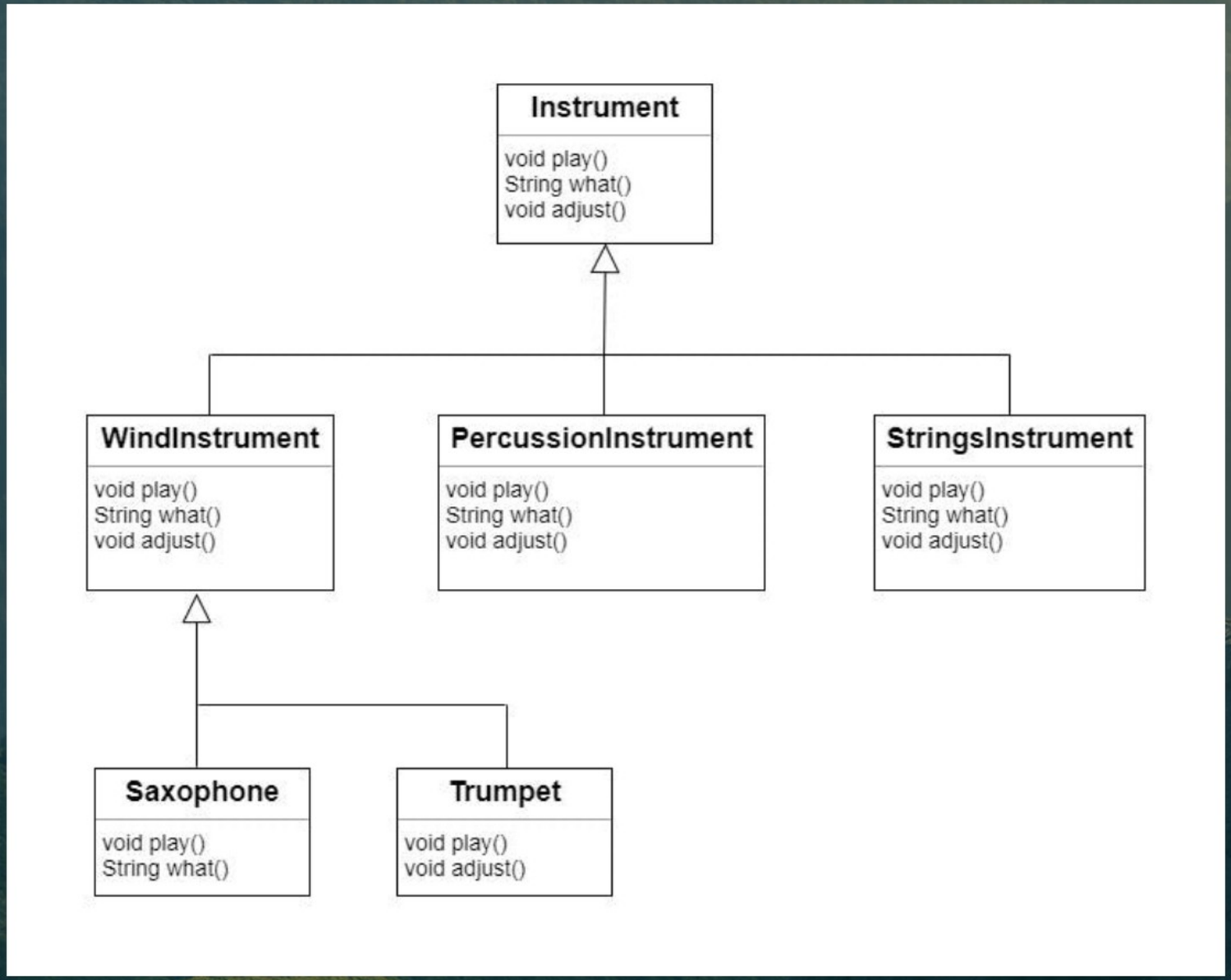

Mentimeter
Laborator 5
Abstractizare

Mentimeter



Polimorfism

2 1


Mentimeter
Principii POO de bază
→ Abstractizare
→ Încapsulare
→ Moștenire
→ Polimorfism
1
1
1


Mentimeter
Abstractizare
→ construirea unui șablon general pe care se bazează o anumită categorie de produse
→ obiectele cunosc interfața, dar nu concret și implementarea
3

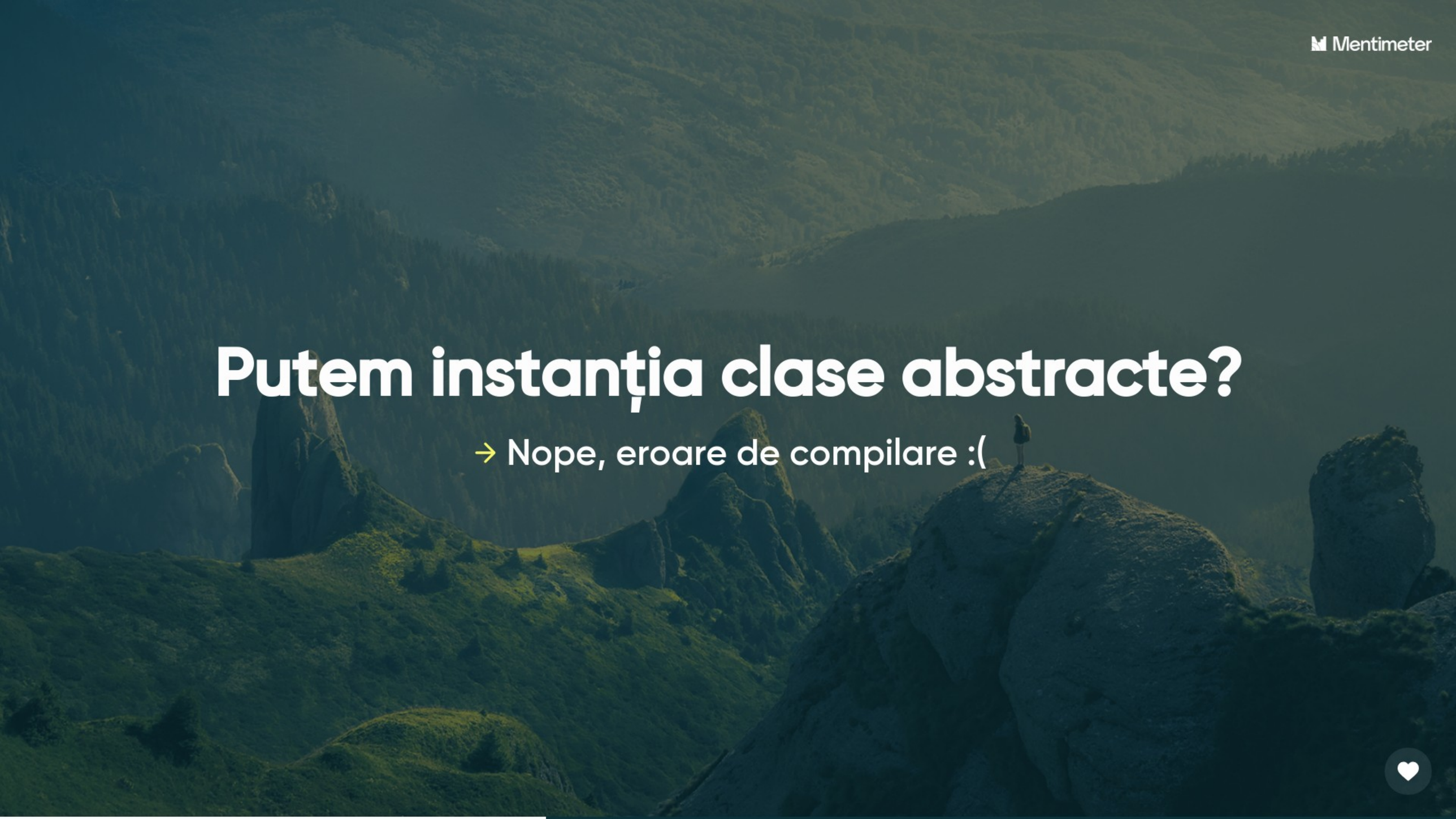

Mentimeter
Clase abstracte
→ vrem să implementăm doar o parte din metode
→ clasa nu poate fi instanțiată
?


Mentimeter
Interfețe
→ avem doar o descriere a structurii, fără implementări
→ o clasă poate extinde mai multe interfețe, dar nu poate moșteni mai multe clase
1
?


Mentimeter
Clase abstracte
→ ce au în comun toate clasele derivate?
→ ce funcționalitate diferă = metode abstracte
1

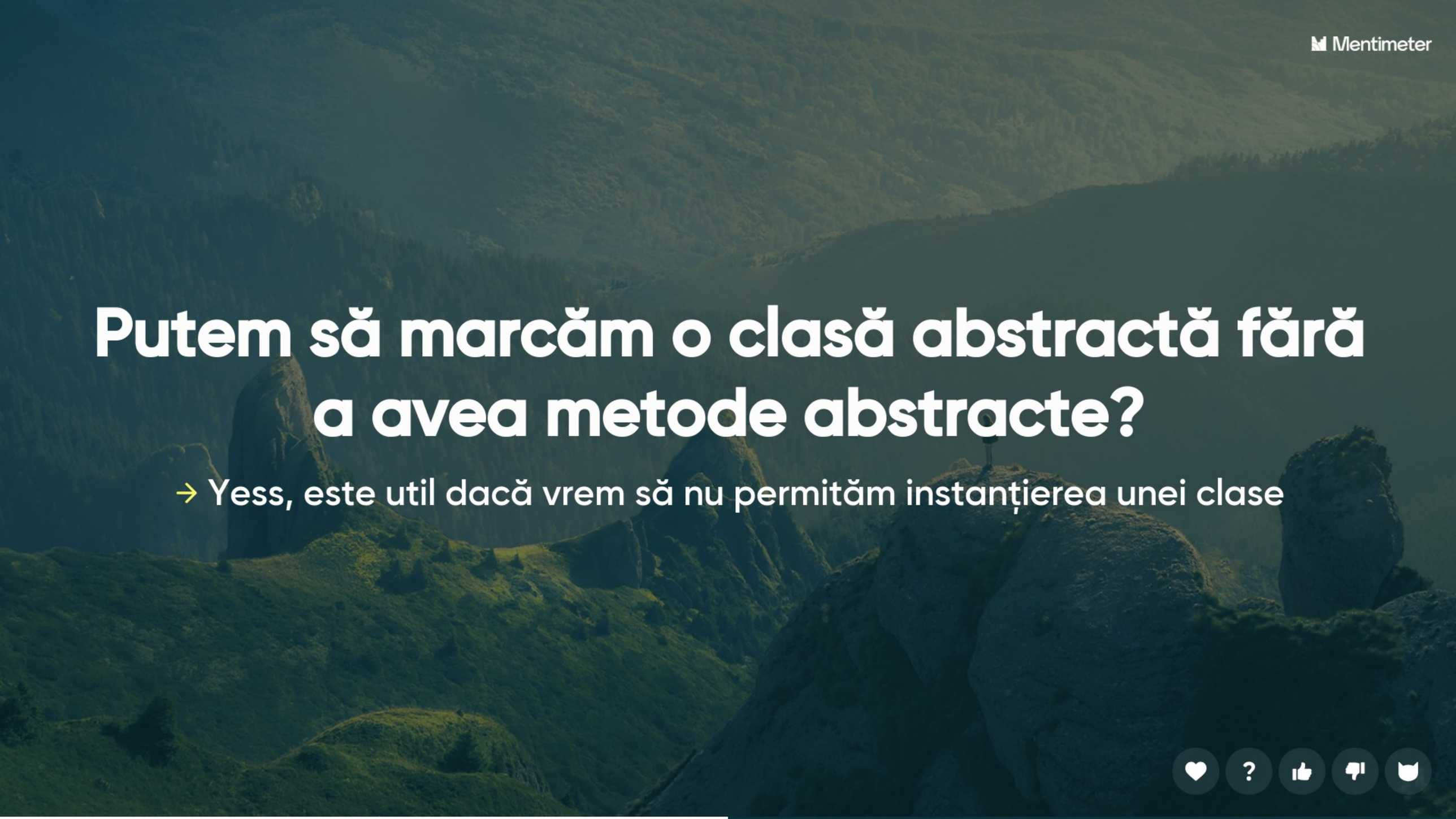

Mentimeter
Când folosim clase abstracte?
→ vrem o interfață comună pentru un set de clase
→ reutilizăm o serie de metode și membrii din această clasă în clasele derivate
1
1
?


Mentimeter
Dynamic binding
→ compilatorul determină implementarea potrivită la run time
→ la baza polimorfismului


Mentimeter
Putem instanția clase abstracte?
→ Nope, eroare de compilare :(


Mentimeter
Metode abstracte
→ abstract return_type function_name(parameters);
→ o clasă cu cel puțin o metodă abstractă = clasă abstractă


Mentimeter
Moștenirea claselor abstracte
→ dacă nu implementăm toate metodele abstracte => o altă clasă abstractă
2


Mentimeter
Putem să marcăm o clasă abstractă fără a avea metode abstracte?
→ Yess, este util dacă vrem să nu permitățm instanțierea unei clase
?

Mentimeter

```java
abstract class Human {
    String name;

    abstract void getJob();

    public void callMe() {
        System.out.println("hello " + name);
    }
}

class Student extends Human {
    void getJob() {
        System.out.println("Internship");
    }
}
```

Exemplu clasa abstracta

1


Mentimeter
Interfețe
→ nume de metode, listă de argumente, valori întoarse daar fără implementare
→ pot conține câmpuri care by default sunt final & static & public
→ toate metodele declarate sunt public implicit
1
?

Mentimeter

Crearea unei interfețe

- cuvântul cheie interface în loc de class
- interfață publică => trebuie să aibă același nume cu fișierul
- fără specificator de acces => package-private


Mentimeter
Moștenirea unei interfețe
→ de către o clasă: implements
→ de către o interfață: extends


Mentimeter
Metodele sunt package-private by default
→ trebuie marcate ca public dacă vrem să fie așa

Mentimeter

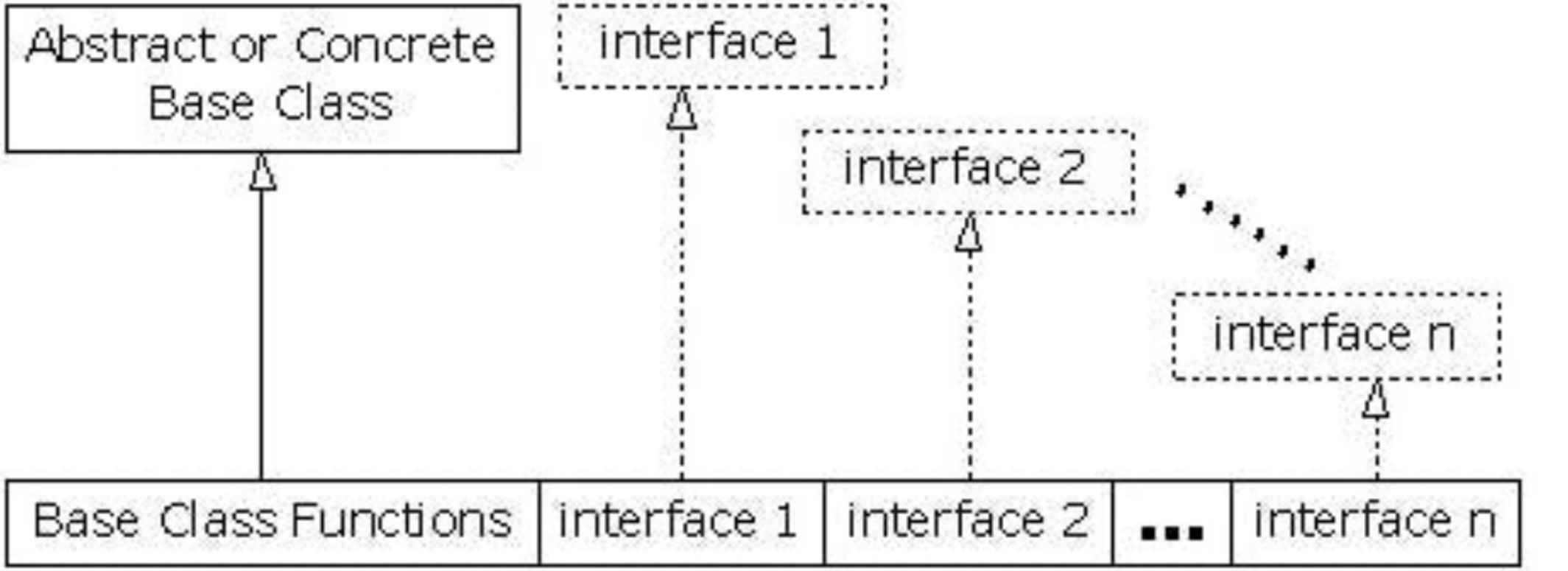
```java
interface Human {
    boolean isAlive = true;
}

interface Student extends Human {
    public void cry();
}

class StudentLaPoli implements Student {
    public void cry() {
        System.out.println("crying in deadlines");
    }
}
```

Exemplu interfețe

Mentimeter



Moștenire multiplă în Java

Mentimeter

```java
interface CanFight {
    void fight();
}

interface CanSwim {
    void swim();
}

interface CanFly {
    void fly();
}

class ActionCharacter {
    public void fight() {}
}

class Hero extends ActionCharacter implements CanFight, CanSwim, CanFly {
    public void swim() {}
    public void fly() {}
}
```

Exemplu 1

Mentimeter

```
class Adventure {
    static void t(CanFight x) {
        x.fight();
    }

    static void u(CanSwim x) {
        x.swim();
    }

    static void v(CanFly x) {
        x.fly();
    }

    static void w(ActionCharacter x) {
        x.fight();
    }

    public static void main(final String[] args) {
        Hero hero = new Hero();
        t(hero);
        u(hero);
        v(hero);
        w(hero);
    }
}
```

Exemplu 2


Mentimeter
Metode default & statice în interfețe

Mentimeter

```java
interface Test {
    default void print() {
        System.out.println("Default print");
    }
}

class CustomTest implements Test {
    void test() {
        print();
    }
}
```

Exemplu metodă default

Mentimeter

```java
interface Test {
    static void print() {
        System.out.println("Default print");
    }
}

class CustomTest implements Test {
    void test() {
        Test.print();
    }
}
```

Exemplu metodă static

1